

කාල සැකසීම (Scheduling)

Author: Yasithzz | 2026

03 -- කාල සැකසීම (Scheduling)

LifeMate ව්‍යාපෘතිය -- සිංහල ලේඛනය

1. Scheduling (කාල සැකසීම) යනු කුමක්ද?

LifeMate ව්‍යාපෘතියේ Scheduling (කාල සැකසීම) feature -- ML model ලිපි කරන stress level (ආතතිය) අනුව, පරිශීලකයාට ස්වයංක්රීයව සතිකාල සැකසීමක් (Weekly Schedule) සකස් කිරීමේ ක්‍රියාවලියයි. ලිපිල් ලෙස -- "ඔබේ stress High නම්, අද වියායාම කළේ කරමු, rest time දිගු කරමු" -- ආකාරයෙන් දිනපතා සැකසීම ස්වයංක්රීයව ගොඩනැගෙයි.

2. WeeklyScheduleService.java -- රේඛා රේඛා පැහැදිලි කිරීම

2.1 -- Package සහ Imports

```
package com.lifemate.backend.service;
import com.lifemate.backend.model.HolidayLeave;
import com.lifemate.backend.model.User;
import com.lifemate.backend.model.WeeklySchedule;
import com.lifemate.backend.model.WeeklySchedule.DaySlot;
import com.lifemate.backend.repository.HolidayLeaveRepository;
import com.lifemate.backend.repository.WeeklyScheduleRepository;
```

සිංහල: Service class -- ශිෂ්ටිකරණ ශීර්ෂ (business logic) ගනු ලබයි. WeeklySchedule model, HolidayLeave model, සහ ඒ MongoDB repository ගනු ලබයි.

2.2 -- Service Class සහ Constructor

```
@Service
public class WeeklyScheduleService {
    private final WeeklyScheduleRepository repo;
    private final HolidayLeaveRepository hlRepo;

    public WeeklyScheduleService(WeeklyScheduleRepository repo, HolidayLeaveRepository hlRepo) {
        this.repo = repo;
        this.hlRepo = hlRepo;
    }
}
```

සිංහල:

- * @Service -- Spring Boot dependency injection.
 - * repo -- WeeklySchedule MongoDB database ශීර්ෂ.
 - * hlRepo -- HolidayLeave (නිවාඩු/ලිව්) database ශීර්ෂ.
-

2.3 -- generateForCurrentWeek()

```
public WeeklySchedule generateForCurrentWeek(String userEmail, int stressIndex,
                                             String stressLevel, User user) {
    LocalDate monday = mondayOfCurrentWeek();
    repo.deleteAllByUserEmailAndWeekStartDate(userEmail, monday.format(FMT));
    return buildAndSave(userEmail, stressIndex, stressLevel, user, monday);
}
```

සිංහල:

- * මෙම ශීර්ෂය ඊළඟ සතිය ආරම්භ කළ (Monday) ස්ථානය ගනී.
 - * ඒ සතිය ගණනය කළ (duplicate) schedule delete කරයි.
 - * buildAndSave() -- නව schedule නිර්මාණය හා MongoDB ලිපි save.
 - * LifestylePage Submit කළ විට backend trigger.
-

2.4 -- getLatestOrGenerate()

```
public WeeklySchedule getLatestOrGenerate(String userEmail, int stressIndex,
                                           String stressLevel, User user) {
    LocalDate monday = mondayOfCurrentWeek();
    List<WeeklySchedule> existing =
        repo.findByUserEmailAndWeekStartDate(userEmail, monday.format(FMT));
    if (!existing.isEmpty()) {
        if (existing.size() > 1) {
            existing.stream().skip(1).forEach(repo::delete);
        }
        return existing.get(0);
    }
    return buildAndSave(userEmail, stressIndex, stressLevel, user, monday);
}
```

සිංහල:

- * ඉදිරිය සතිය schedule ඇත් නම් ඒ return කරයි.
 - * Duplicates ඇත් නම් (crash recovery) -- extras delete, ප්‍රථම record return.
 - * Schedule නොමැති නම් -- buildAndSave() නව schedule සෑදීමයි.
-

2.5 -- updateSlotStatus()

```

public WeeklySchedule updateSlotStatus(String userEmail, String day,
                                     String itemId, String status) {
    WeeklySchedule ws = repo.findTopByUserEmailOrderByGeneratedAtDesc(userEmail)
        .orElseThrow() -> new RuntimeException("No schedule found");
    List<DaySlot> slots = ws.getDays().getOrDefault(day, List.of());
    slots.forEach(s -> { if (s.getItemId().equals(itemId)) s.setStatus(status); });
    ws.setUserModified(true);
    return repo.save(ws);
}

```

සිංහල: Schedule page හි user "Complete" button ක්ලික් කළ විට -- ඒ slot (activity) pending -> completed ලෙස update. Database update ශීර්ෂය.

2.6 -- addCustomSlot()

```

public WeeklySchedule addCustomSlot(String userEmail, String day, DaySlot slot) {
    WeeklySchedule ws = repo.findTopByUserEmailOrderByGeneratedAtDesc(userEmail)
        .orElseThrow() -> new RuntimeException("No schedule found");
    slot.setItemId(UUID.randomUUID().toString());
    ws.getDays().computeIfAbsent(day, k -> new ArrayList<>()).add(slot);
    sortDay(ws.getDays().get(day));
    ws.setUserModified(true);
    return repo.save(ws);
}

```

සිංහල: User schedule page හි "Add Activity" button ක්ලික් කළ විට -- custom activity schedule ලෙස add. UUID ලෙස unique ID ලබා දෙයි. sortDay() කාල ලෙස sorted.

2.7 -- removeSlot()

```

public WeeklySchedule removeSlot(String userEmail, String day, String itemId) {
    WeeklySchedule ws = repo.findTopByUserEmailOrderByGeneratedAtDesc(userEmail)
        .orElseThrow() -> new RuntimeException("No schedule found");
    ws.getDays().getOrDefault(day, List.of()).removeIf(s -> s.getItemId().equals(itemId));
    ws.setUserModified(true);
    return repo.save(ws);
}

```

සිංහල: Slot delete button ක්ලික් කළ විට -- ඒ itemId ගලපා list ලෙස remove කරයි.

2.8 -- buildAndSave() -- ජර්ධාන Schedule ගොඩනැගීම

```
private WeeklySchedule buildAndSave(String userEmail, int stressIndex, String stressLevel,
    User user, LocalDate monday) {
    List<String> weekDates = new ArrayList<>();
    for (int i = 0; i < 7; i++) weekDates.add(monday.plusDays(i).format(FMT));
    Set<String> leaveDates = new HashSet<> (
        hlRepo.findByUserEmailOrderByDateAsc(userEmail).stream()
            .map(h -> h.getDate()).filter(weekDates::contains).toList());
```

සිංහල:

- * සතිය (Mon-Sun) 7 දිනක්ගේ ස්ථාන ගනී.
- * ඒ user ගේ holidays/leave dates MongoDB ලෙස ගනී -- ශ්රේණි ද් ලෙස filter.

```
String[] DAYS = {"MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY", "SATURDAY", "SUNDAY"};
Set<String> workingDaySet = new HashSet<> (
    user.getWorkingDays() != null && !user.getWorkingDays().isEmpty()
        ? user.getWorkingDays()
        : List.of("MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY"));

for (int i = 0; i < 7; i++) {
    boolean isLeave = leaveDates.contains(dateStr);
    boolean isWorkingDay = workingDaySet.contains(DAYS[i]);
    ws.getDays().put(DAYS[i], isLeave
        ? leaveSlots()
        : buildDay(stressIndex, !isWorkingDay, user));
}
```

සිංහල:

- * User profile ලෙස working days set කළ නම් ඒ ශ්රේණි ද් ලෙස ගනී.
- * Default: Monday-Friday working days.
- * Leave/Holiday -- leaveSlots() (Rest Day schedule).
- * Working/Weekend -- buildDay() (stress අනුව schedule).

2.9 -- buildDay() -- Stress Level අනුව Schedule

```
private List<DaySlot> buildDay(int stress, boolean weekend, User user) {
    String wake = nvl(user.getWakeTime(), "07:00");
    String bed = nvl(user.getSleepSchedule(), "22:30");
    String[] wWork = parseWorkHours(nvl(user.getWorkHours(), "09:00-17:00"));
    String wp = nvl(user.getWorkoutPreference(), "Moderate");
    String ft = nvl(user.getFreeTimePreference(), "Reading");
    String mp = nvl(user.getMealPreference(), "Balanced");
```

සිංහල: User profile (wake time, bedtime, workout preference, meal preference) ගනනෙයි. Default values ද සකස් කරනේ.

Stress Level අනුව Activities:

stress	Work Blocks	Break Time	Exercise
0-1 (Very Low/Low)	90 min blocks	10 min breaks	Full intensity, 40-50 min
2 (Normal)	90 min blocks	10 min breaks	Moderate, 40 min
3 (High)	75 min blocks	15 min breaks	Light walk, 30 min
4+ (Very High)	45 min blocks	20 min breaks	Gentle yoga only

```
int blockLen = stress >= 4 ? 45 : stress == 3 ? 75 : 90;
int breakLen = stress >= 4 ? 20 : stress == 3 ? 15 : 10;
```

සිංහල: Stress ඉහළ නම් -- work blocks කවේව, breaks දිගු ලෙස. Stress අඩු නම් -- focus blocks දිගු ලෙස, breaks කවේව.

```
int mrMins = stress >= 3 ? 30 : 20;
s.add(slot(wake, addMins(wake, mrMins), "Other", "Morning Routine & Freshen Up", "pending"));
```

සිංහල: High stress: morning routine 30 min. Low stress: 20 min. Stress ඉහළ නම් ශරීරය සුදානම් කිරීමේ කාලය දිගු.

2.10 -- Helper Methods

```
private String addMins(String time, int mins) {
    int[] t = parseTime(time);
    int total = t[0]*60 + t[1] + mins;
    return fmt(total/60, total%60);
}

private String subtractMins(String time, int mins) {
    int[] t = parseTime(time);
    int total = Math.max(0, t[0]*60 + t[1] - mins);
    return fmt(total/60, total%60);
}
```

සිංහල: Time string ("07:00") ශීර්ෂීඨ ලෙස minutes add/subtract. LifeMate schedule slot start/end times ගණනය.

```
private String exerciseName(String pref, int stress) {
    if (stress >= 4) return "Gentle Yoga / Breathing Exercises";
    if (stress == 3) return "Light Walk or Stretching";
    return switch (pref.toLowerCase()) {
        case "intense" -> "High-Intensity Workout (HIIT / Running)";
        case "light" -> "Light Walk or Yoga";
        default -> "Moderate Exercise (Cycling / Gym)";
    };
}
```

සිංහල: Stress level + user workout preference අනුව exercise ශීර්ෂයේ ලෙස select. High stress -- gentle yoga. User "Intense" preference -- HIIT.

```
private String mealName(String pref, String mealType) {
    String base = switch (pref.toLowerCase()) {
        case "vegetarian", "vegan" -> "Plant-Based ";
        case "high-protein" -> "High-Protein ";
        case "keto" -> "Keto ";
        case "mediterranean" -> "Mediterranean-Style ";
        default -> "";
    };
    return base + mealType;
}
```

සිංහල: User meal preference (vegetarian, keto, etc.) අනුව Breakfast/Lunch/Dinner label personalize. "Keto Breakfast", "Plant-Based Lunch" ආකාරයෙන්.

2.11 -- mondayOfCurrentWeek()

```
private LocalDate mondayOfCurrentWeek() {
    LocalDate today = LocalDate.now();
    return today.with(DayOfWeek.MONDAY);
}
```

සිංහල: ඊළඟ (current) සතිය Monday දිනය ගනී. Schedule සදා ගැනීමේ starting point.

3. ScheduleRepository.java -- පැහැදිලි කිරීම

```
public interface ScheduleRepository extends MongoRepository<Schedule, String> {
    Optional<Schedule> findTopByUserEmailOrderByCreatedAtDesc(String userEmail);
    Optional<Schedule> findByUserEmailAndScheduleDate(String userEmail, String scheduleDate);
    void deleteByUserEmail(String userEmail);
}
```

සිංහල: MongoDB query methods -- Spring Data ස්වයංක්රීයව SQL-free query generate.

Method	ප්රයෝජනය
`findTopByUserEmailOrderByCreatedAtDesc`	User ගේ ලෝකයේ නවතම schedule ගැනීම
`findByUserEmailAndScheduleDate`	Specific date schedule ගැනීම
`deleteByUserEmail`	User account delete -- schedule ද delete

4. Scheduling Flow -- සම්පූර්ණ ක්රියාවලිය

```
User -> LifestylePage (8 lifestyle inputs submit)
v
Backend AuthService -> ML Service POST /predict
v
ML Service -> stress_level, stress_index return
v
Backend WeeklyScheduleService.generateForCurrentWeek()
v
1. mondayOfCurrentWeek() -- සතිය Monday ගනී
2. HolidayLeave MongoDB -- leave dates ගනී
3. User profile (wake, sleep, workout, meal) ගනී
4. 7 දිනක් loop: buildDay(stressIndex, isWeekend, user)
   - Stress level අනුව work/break/exercise slots
   - User preferences (meal, workout) personalize
5. WeeklySchedule MongoDB save
v
Frontend SchedulePage -- GET /api/schedule/weekly
v
User screen -- Mon to Sun slots display
```

ලේඛනය -- LifeMate ව්‍යාපෘතිය, Yasithzz, 2026